

Research Pipeline: Evaluating Recommender Systems under Sparse User Feedback

Kamal Lalloo

September 2026

1 Introduction

This document records the implemented experimental pipeline used to evaluate recommender systems under progressively sparse user feedback. It is intended as a compact technical reference for the dissertation and reflects the final code and configurations rather than the earlier proposal. The study uses MovieLens-1M and Amazon Reviews 2023 Video Games, converts explicit ratings into positive implicit interactions, applies iterative 5-core filtering, constructs chronological leave-one-out targets, sparsifies only the training histories, and evaluates ten recommender models using a standardized RecBole pipeline.

The design follows the broader recommendation-evaluation literature emphasizing controlled preprocessing, consistent splitting, careful hyperparameter tuning, strong baselines, and reproducible evaluation [5, 6, 11]. RecBole is used as the common training and evaluation framework [8, 9].

2 High-Level Pipeline

The complete experimental workflow is:

1. Acquire MovieLens-1M and Amazon Reviews 2023 Video Games.
2. Convert ratings ≥ 4 into positive implicit-feedback interactions.
3. Deduplicate repeated Amazon positive user-item interactions.
4. Apply iterative 5-core filtering to users and items.
5. Deterministically order each user’s interactions chronologically and create `sequence_order`.
6. Define chronological leave-one-out targets: final interaction for test, second-last for validation, and all earlier interactions for training.
7. Apply controlled sparsity only to the training split.
8. Recombine sparse training data with the original validation and test interactions.
9. Convert each condition to RecBole `.inter` format.
10. Write the same fixed `.item` catalogue to the baseline and every sparsity condition.
11. Tune model hyperparameters only on the full baseline for each dataset.
12. Freeze the selected configuration and train each model across all sparsity conditions.
13. Evaluate with full-sort ranking using Recall, NDCG, Hit Rate and MRR at $K \in 5, 10, 20$.
14. Process results and compare both absolute performance and degradation relative to the baseline.

The pipeline deliberately changes the amount and location of *training evidence* while keeping the evaluation targets and candidate catalogue stable. This isolates the effect of sparse feedback

more cleanly than allowing the validation/test targets or candidate universe to change between conditions.

3 Datasets

MovieLens-1M is a standard recommender-system benchmark containing timestamped user ratings [1]. Amazon Reviews 2023 is a substantially larger and naturally sparse collection of product interactions; the Video Games category is used here [2, 3]. For Amazon, `parent_asin` is used as the item identifier.

3.1 Preprocessing Statistics

Table 1: Dataset size through the implemented preprocessing pipeline.

Stage	MovieLens-1M			Amazon Video Games		
	Users	Items	Interactions	Users	Items	Interactions
Raw	6,040	3,706	1,000,209	2,766,656	137,249	4,624,615
Positive ($r \geq 4$)	6,038	3,533	575,281	2,115,867	120,360	3,445,132
After Amazon deduplication	–	–	–	2,115,867	120,360	3,394,554
Final iterative 5-core	6,034	3,125	574,376	63,413	19,022	533,550

Table 2: Final chronological leave-one-out splits and user-history statistics.

Statistic	MovieLens-1M	Amazon Video Games
Training interactions	562,308	406,724
Validation interactions	6,034	63,413
Test interactions	6,034	63,413
Mean full 5-core history	95.19	8.41
Median full 5-core history	58	6
Minimum full 5-core history	5	5
Maximum full 5-core history	1,417	314

The two datasets therefore provide substantially different interaction regimes. MovieLens is comparatively dense, whereas Amazon has very short user histories even after 5-core filtering. The Amazon core is also a much smaller, more active subset of the raw population; this should be acknowledged when discussing external validity.

4 Preprocessing

4.1 Positive Implicit Feedback

Both datasets are converted to positive implicit feedback by retaining ratings greater than or equal to four. The remaining rating magnitude is discarded: an observed row represents a positive user–item interaction. This formulation is consistent with implicit-feedback ranking, where the task is to rank relevant items from observed and unobserved candidates rather than predict explicit rating values [4, 14].

The rating threshold is applied before 5-core filtering. DaisyRec 2.0 provides direct precedent for treating MovieLens ratings of at least four as positive in rigorous top- N evaluation and emphasizes that preprocessing choices can materially change experimental results [6].

4.2 Amazon Deduplication

Amazon contains repeated positive interactions between the same user and `parent_asin`. After thresholding, repeated positive user-item rows are sorted by timestamp and original source-row order, and only the most recent positive interaction is retained. A total of 50,578 repeated positive rows are removed.

This produces one positive implicit interaction per user-item pair and avoids repeated reviews of the same product being interpreted as independent item-discovery targets. The rule is project-specific: it should be documented explicitly rather than presented as a universal preprocessing standard. The official Amazon Reviews 2023 benchmark resources likewise provide preprocessing and recommendation splits, but exact duplicate rules may differ [3].

4.3 Iterative 5-Core Filtering

A 5-core is applied *after* positive-feedback conversion and, for Amazon, after deduplication. Users and items with fewer than five retained positive interactions are repeatedly removed until all remaining users and items satisfy the threshold.

The filtering is iterative because removing low-frequency items can cause a user to fall below the user threshold, and vice versa. Core filtering is common in recommendation benchmarking, but the chosen threshold changes the evaluation population and should therefore be reported explicitly [6, 25]. The 5-core is applied only to construct the stable baseline dataset; sparse training conditions are *not* required to remain 5-core.

4.4 Deterministic Temporal Ordering

Each user’s interactions are sorted by timestamp. When timestamps are tied, original source-row order is used as a deterministic tie-breaker. A new integer field, `sequence_order`, is then assigned as $1, \dots, N_u$ for each user.

This field becomes the RecBole `TIME_FIELD`. Preserving a deterministic user-level ordering is particularly important for sequential recommenders such as GRU4Rec, SASRec and BERT4Rec [18, 19, 20]. The tie-breaker guarantees reproducibility, although the imposed order between events sharing the same timestamp is not necessarily the true real-world event order.

4.5 Chronological Leave-One-Out Split

For user u with ordered sequence

$$(i_1, i_2, \dots, i_{N_u}),$$

the implemented split is

$$\text{train}_u = (i_1, \dots, i_{N_u-2}), \quad \text{validation}_u = i_{N_u-1}, \quad \text{test}_u = i_{N_u}.$$

The manual preprocessing split first defines immutable validation and test targets. Later, after training sparsity is applied, sparse training interactions are recombined with these two original

targets and RecBole re-creates the same leave-one-out split using `sequence_order`. Validation checks confirm that the final two interactions remain unchanged.

Leave-one-out preserves each user’s local chronology and aligns naturally with next-item prediction. It does not, however, impose a single global time boundary across all users; recent work shows that global temporal splitting can produce different model rankings and may better reflect some deployment settings [24]. This is best treated as a limitation rather than a reason to change completed experiments.

5 Controlled Sparsity Simulation

Sparsity is applied *only* to the training split. Validation and test interactions remain unchanged for every condition.

For a user with n_u training interactions and requested retention fraction p , the number retained is

$$m_u(p) = \max(1, \lceil n_u p \rceil). \quad (1)$$

The ceiling rule guarantees at least one retained training interaction per user. Consequently, 50%, 25% and 10% are *requested or nominal* retention levels; realized dataset-wide retention can be higher, especially for users with short histories. This is particularly relevant to Amazon, where the median full 5-core history is only six interactions.

5.1 Random Per-User Sparsity

The repository retains the historical folder name `global`, but the implemented academic meaning is **random per-user retention**, not unrestricted matrix-global deletion.

For each user, a stable user-specific seed is derived from the fixed sparsity seed 2025 and the user identifier using SHA-256. The user’s training interactions are deterministically shuffled, the first $m_u(p)$ are selected, and the surviving interactions are returned to chronological order.

Because the same deterministic shuffle is reconstructed for every retention level,

$$S_u(10\%) \subseteq S_u(25\%) \subseteq S_u(50\%) \subseteq S_u(100\%).$$

The nested design reduces unnecessary variation between sparsity levels. The exact nested, per-user construction is a project design choice rather than a literature standard.

5.2 Recent-History Sparsity

For each user, only the most recent $m_u(p)$ training interactions are retained. This evaluates performance when the model has a short but temporally recent behavioral history.

5.3 Early-Profile Sparsity

For each user, only the earliest $m_u(p)$ training interactions are retained. This is best described as **cold-start-like** or **limited early-profile** evaluation rather than true cold start, because the users remain known and retain at least one training interaction.

The distinction between removing information from different temporal locations is meaningful for sequential recommendation: Betello et al. show that sequence position can materially affect robustness when training interactions are perturbed [22].

5.4 Experimental Conditions

The final analysis uses ten conditions per dataset:

- one full-data baseline;
- random per-user retention at 50%, 25%, and 10%;
- recent-history retention at 50%, 25%, and 10%;
- early-profile retention at 50%, 25%, and 10%.

The generated 100% versions of the three sparsity scenarios are used as identity controls: they must reproduce the original baseline training data exactly.

6 RecBole Dataset Construction and Evaluation Universe

Each condition is exported as one RecBole interaction file containing sparse training interactions plus the unchanged validation and test targets. Fields are

```
user_id:token
item_id:token
timestamp:float
sequence_order:float
```

The dataset configuration uses `USER_ID_FIELD=user_id`, `ITEM_ID_FIELD=item_id`, and `TIME_FIELD=sequence_order`. `rm_dup_inter` is disabled because duplicate handling has already been performed explicitly.

A fixed `.item` catalogue is constructed from the complete post-preprocessing temporal dataset and copied identically to the baseline and every sparse condition. This keeps the full-sort candidate universe constant across conditions. Some evaluation targets can therefore remain in the candidate catalogue even if all of their training interactions disappear under strong sparsification. Metadata records these unseen-target cases so they can be considered during interpretation.

Full-sort evaluation is preferable to sampled-negative evaluation when feasible because sampled ranking metrics can be inconsistent with their exact counterparts and can even change relative conclusions between recommenders [7].

7 Evaluation Configuration

Both datasets use the same RecBole evaluation protocol:

Table 3: Shared evaluation configuration.

Setting	Value
Split	leave-one-out: validation and test
Grouping	user
Ordering	temporal (<code>sequence_order</code>)
Evaluation mode	full sort
Metrics	Recall, NDCG, Hit, MRR
Top- K	5, 10, 20
Validation metric	NDCG@10
Higher validation score preferred	true
Metric decimal places	6

With exactly one relevant held-out item per user, $\text{Recall}@K$ and $\text{HitRate}@K$ are mathematically equivalent under the standard leave-one-out definition. They can both be reported for comparability with prior work, but they should not be interpreted as independent evidence. NDCG and MRR additionally capture how highly the target is ranked.

8 Sequence-Length Configuration

MovieLens explicitly sets

$$\text{MAX_ITEM_LIST_LENGTH} = 200.$$

Amazon does not override this setting and therefore uses the RecBole 1.2.1 default of 50 [10]. This difference is reasonable given the much shorter Amazon histories, but it should be reported explicitly.

9 Models

Ten models are evaluated to cover different recommendation paradigms:

Table 4: Selected recommender models.

Family	Model	Reference	Role
Non-personalized	Pop	[12]	Popularity reference baseline.
Neighborhood	ItemKNN	[13]	Classical item-similarity collaborative filtering.
Latent factor	BPR	[14]	Pairwise implicit-feedback ranking baseline.
Linear	EASE	[15]	Strong shallow linear sparse-data model.
Neural	NeuMF	[16]	Neural collaborative filtering.
Neural	MultiVAE	[17]	Variational autoencoder for implicit feedback.
Sequential	GRU4Rec	[18]	Recurrent sequence model.
Sequential	SASRec	[19]	Unidirectional self-attention sequence model.
Sequential	BERT4Rec	[20]	Bidirectional masked-item sequence model.
Graph	LightGCN	[21]	Simplified graph collaborative filtering.

Including strong classical and linear baselines is important because more complex neural recommenders do not automatically outperform well-tuned simpler methods under controlled evaluation [11, 15].

10 Hyperparameter Tuning

Tuning is performed independently for each dataset but *only on the full baseline condition*. Every candidate is trained and compared using validation NDCG@10. The winning baseline configuration is then frozen across all sparse conditions for that dataset.

This design measures robustness to reduced training information without simultaneously adapting hyperparameters to each sparsity level. It is a constrained grid search rather than an exhaustive optimization procedure, consistent with the need to control tuning as part of rigorous recommender evaluation [5, 6].

Table 5: Constrained tuning search space.

Model	Candidate values
ItemKNN	$k \in 50, 100, 200$
EASE	<code>reg.weight</code> $\in 100, 250, 500$
BPR	embedding $\in 32, 64$; weight decay $\in 10^{-6}, 10^{-5}, 10^{-4}$
NeuMF	MF embedding $\in 32, 64$; MLP embedding $\in 32, 64$
MultiVAE	latent dimension $\in 100, 200$; dropout $\in 0.3, 0.5$
GRU4Rec	hidden size $\in 64, 128$; dropout $\in 0.1, 0.2$
SASRec	layers $\in 1, 2$; hidden dropout $\in 0.1, 0.2$
BERT4Rec	layers $\in 1, 2$; hidden dropout $\in 0.1, 0.2$
LightGCN	layers $\in 1, 2, 3$; regularization $\in 10^{-5}, 10^{-4}$
Pop	no tuned hyperparameters

11 Final Model Configurations

Common settings for trainable models are a maximum of 100 epochs, learning rate 0.001, validation every epoch, and early-stopping patience of 10 validation steps. NDCG@10 determines the best checkpoint. Model configuration files specify seed 2025 and `reproducibility: true`; run metadata also records the model seed and software versions.

Table 6: Selected dataset-specific model configurations.

Model	MovieLens-1M	Amazon Video Games
Pop	1 epoch	1 epoch
ItemKNN	$k = 50$, item-based, shrink 0	$k = 200$, item-based, shrink 0
BPR	emb. 64, weight decay 10^{-6}	emb. 64, weight decay 10^{-6}
EASE	reg. 500	reg. 100
NeuMF	MF 64, MLP 32, hidden [64,32,16,8]	MF 64, MLP 64, hidden [64,32,16,8]
MultiVAE	latent 200, dropout 0.5, hidden [600]	latent 200, dropout 0.5, hidden [600]
GRU4Rec	emb. 64, hidden 128, 1 layer, drop 0.2	emb. 64, hidden 64, 1 layer, drop 0.2
SASRec	2 layers, hidden 64, 2 heads, hidden drop 0.1	1 layer, hidden 64, 2 heads, hidden drop 0.2
BERT4Rec	2 layers, hidden 64, 2 heads, hidden drop 0.1	2 layers, hidden 64, 2 heads, hidden drop 0.2
LightGCN	emb. 64, 1 layer, reg. 10^{-5}	emb. 64, 2 layers, reg. 10^{-5}

For both SASRec and BERT4Rec, inner size is 256, attention dropout is 0.2, and the number of attention heads is two. BERT4Rec uses mask ratio 0.2. Cross-entropy sequential models disable training negative sampling. BPR retains pairwise negative-sampling behavior appropriate to its objective.

12 BERT4Rec Compatibility Patch

The project uses RecBole 1.2.1. BERT4Rec is executed through a project-local subclass, `PatchedBERT4Rec`. The patch changes only cross-entropy target-validity selection.

The relevant distinction is that sequence position 0 is a valid location, whereas RecBole item ID 0 is reserved for padding. The project patch therefore identifies real masked targets using

`pos_items > 0`

instead of treating

`masked_index > 0`

as the validity condition. Architecture, forward propagation, masking structure, prediction and evaluation remain inherited from RecBole. The patch explicitly supports CE loss and is covered by a dedicated unit test.

Because implementation details can materially affect recommender results, the run log records the RecBole version and whether the local BERT4Rec patch was active [9].

13 Training and Final Evaluation

For each final run, `train_model.py`:

1. parses the dataset, model, condition, run type, device request and optional seed;
2. validates the required `.inter` and `.item` files;
3. loads `dataset.yaml`, `evaluation.yaml`, and the model YAML;
4. overrides the dataset path with the selected baseline/sparsity directory;
5. creates the RecBole dataset and train/validation/test DataLoaders;
6. resolves the requested model class, using the local BERT4Rec subclass where required;
7. initializes the appropriate RecBole trainer;
8. trains with validation-based checkpoint selection;
9. evaluates the saved best-validation checkpoint on the test set;
10. writes metrics and experimental provenance to `results/raw/experiment_results.csv`.

The saved provenance includes the dataset condition, model seed, sparsity seed, requested and realized retention, RecBole/PyTorch/Python versions, Git commit, hashes of the YAML configuration and dataset files, BERT4Rec-patch status, sequence-length configuration, timing information and validation/test metrics.

14 Codebase Execution Order

Table 7: Main code files in pipeline order.

File	Purpose
<code>preprocess_movielens.py</code>	Loads MovieLens ratings, validates raw data, keeps ratings ≥ 4 , applies iterative 5-core filtering, creates deterministic temporal order and leave-one-out splits, and writes preprocessing summaries.
<code>preprocess_amazon.py</code>	Loads Amazon JSONL reviews, maps <code>parent_asin</code> to item ID, keeps ratings ≥ 4 , removes repeated positive user-item interactions, applies iterative 5-core filtering, creates temporal order and leave-one-out splits, and writes summaries.
<code>prepare_recbole_dataset.py</code>	Converts the complete MovieLens temporal dataset to the baseline RecBole <code>movielens.inter</code> format.
<code>prepare_amazon_recbole_dataset.py</code>	Converts the complete Amazon temporal dataset to the baseline RecBole <code>amazon.inter</code> format.

File	Purpose
<code>sparsity.py</code>	Shared implementation of random per-user, recent-history and early-profile retention, including ceiling retention, minimum one interaction, stable user-specific random seeds and nested random subsets.
<code>generate_sparsity_datasets.py</code>	Generates MovieLens sparse training conditions, recombines them with fixed validation/test targets, validates identity/preservation properties, exports RecBole files and metadata.
<code>generate_amazon_sparsity_datasets.py</code>	Amazon counterpart of the MovieLens sparsity generator.
<code>prepare_movielens_item_catalog.py</code>	Builds one fixed MovieLens item catalogue from the complete post-preprocessing dataset and copies it identically to all conditions.
<code>prepare_amazon_item_catalog.py</code>	Builds and verifies the fixed Amazon item catalogue.
<code>verify_datasets.py</code> / <code>verify_amazon_datasets.py</code>	Validate generated dataset structure, metadata, fixed catalogue coverage, split preservation and nested sparsity properties.
<code>tune_model.py</code>	Runs baseline-only constrained grid search. Candidates are compared by validation NDCG@10 and tuning results are kept separate from final experiment results.
<code>extract_best_tuning_configs.py</code>	Extracts the best baseline hyperparameter candidate for each completed dataset-model tuning run.
<code>bert4rec_patch.py</code>	Defines the project-local BERT4Rec CE target-validity fix used by final and tuning runs.
<code>train_model.py</code>	Common final experiment runner for all datasets, conditions and models; handles configuration, RecBole dataset creation, training, evaluation and provenance logging.
<code>process_results.py</code>	Normalizes raw output, filters final runs, checks completeness, computes degradation measures and writes processed tables for analysis.
<code>01_data_exploration.ipynb</code> – <code>06_cross_dataset_analysis.ipynb</code>	Exploration, preprocessing checks, result verification, baseline analysis, sparsity analysis and cross-dataset interpretation.

15 Automated Checks and Reproducibility

The repository includes explicit validation throughout the pipeline rather than relying only on successful execution. Important checks include:

- no missing required values or unexpected duplicate interactions;
- every final 5-core user and item has at least five positive interactions;
- timestamps are non-decreasing within each user’s ordered sequence;
- `sequence_order` is unique and follows $1, \dots, N_u$ before sparsification;
- one validation and one test interaction are retained per user;
- all sparse conditions modify training data only;
- 100% sparse controls are exactly identical to the baseline;
- random per-user subsets are deterministic and nested;

- every user retains at least one sparse training interaction;
- all baseline/sparse item catalogues are identical and cover interaction items;
- the BERT4Rec patch preserves position-zero targets while excluding padding.

Dedicated tests include `test_sparsity.py`, `test_catalogues.py`, and `test_bert4rec_patch.py`. The final runner additionally stores Git, software-version and SHA-256 provenance. This follows the general recommendation-evaluation principle that implementation, preprocessing, tuning and evaluation choices should be controlled and recorded because they can alter conclusions [5, 6, 8].

16 Result Processing and Robustness Analysis

Absolute ranking performance should remain the primary evidence. Robustness is additionally summarized by relative degradation from the full-data baseline:

$$D_{m,c} = \frac{S_{m,\text{full}} - S_{m,c}}{S_{m,\text{full}}}, \quad (2)$$

where $S_{m,\text{full}}$ is model m 's score on the full baseline and $S_{m,c}$ is its score under sparse condition c . Smaller $D_{m,c}$ indicates lower relative degradation.

Relative degradation should always be interpreted together with the underlying absolute score: a weak baseline can display a small percentage drop while remaining worse in absolute recommendation quality. Prior work has directly examined how recommender performance changes as sparsity or sequence perturbations increase [23, 22].

The main analysis should therefore compare:

- absolute NDCG, Recall/Hit and MRR across conditions;
- relative degradation from each model's own baseline;
- differences between random, recent and early-profile sparsity;
- differences between MovieLens and Amazon, especially given their very different history-length distributions;
- whether simpler classical/linear baselines remain competitive with neural, sequential and graph models as feedback is reduced.

17 Methodological Interpretation and Limitations

The completed experiments are methodologically coherent, but the following wording and limitations should be retained in the dissertation:

1. **Random rather than global deletion.** The `global` directory name represents random per-user retention. It is not unrestricted random deletion from the entire interaction matrix.
2. **Nominal retention.** Requested 50%, 25% and 10% levels are not necessarily exact global fractions because of per-user ceiling rounding and the one-interaction minimum.
3. **Early-profile is cold-start-like.** Users remain known and retain training interactions; this is not true unseen-user cold start.
4. **5-core precedes sparsity.** Sparse training sets are intentionally allowed to violate the original 5-core condition.
5. **Fixed evaluation targets.** Validation and test interactions do not change across sparse conditions.
6. **Fixed candidate catalogue.** The item universe is constant, so sparse conditions may contain evaluation items unseen in sparse training. This makes some conditions harder but keeps the candidate task comparable.

7. **Leave-one-out preserves local, not global, chronology.** It is suitable for next-item prediction but does not create a global future-only test period [24].
8. **Timestamp ties.** Source-row tie-breaking is deterministic but may impose an arbitrary order among events with identical timestamps.
9. **Amazon 5-core selection.** The final Amazon benchmark represents more active users/items than the raw population, so conclusions should be scoped to the filtered benchmark.
10. **Recall and Hit redundancy.** With one held-out relevant item per user, Recall@ K and HitRate@ K are equivalent.
11. **One deterministic random sparsity realization.** The fixed sparsity seed improves controlled comparison and nesting, but conclusions about random-interaction removal are based on that constructed realization unless additional sparsity seeds are analyzed.

18 Summary

The final pipeline is a controlled empirical benchmark of recommender robustness to reduced user feedback. Its main methodological strengths are: explicit preprocessing, iterative core filtering, deterministic temporal ordering, immutable leave-one-out targets, training-only sparsification, three interpretable sparsity mechanisms, a fixed candidate catalogue, baseline-only hyperparameter tuning, one common RecBolt evaluation framework, full-sort ranking, strong classical and modern model coverage, and extensive provenance/validation checks.

The central dissertation claim should therefore be framed as an *empirical comparison of robustness to interaction sparsification*, not as a new recommendation algorithm. The contribution is the controlled comparison of how different recommender families degrade when the amount and temporal location of user feedback are systematically restricted.

References

- [1] F. Maxwell Harper and Joseph A. Konstan. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4), 2015.
- [2] Yupeng Hou, Jiacheng Li, Xiangjun Fu, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. Bridging Language and Items for Retrieval and Recommendation: Benchmarking LLMs as Semantic Encoders. In *Proceedings of ACL*, pages 3251–3265, 2026. <https://doi.org/10.18653/v1/2026.acl-long.147>.
- [3] McAuley Lab. Amazon Reviews 2023: Dataset and Recommendation Benchmark Resources. <https://amazon-reviews-2023.github.io/>. Accessed September 2026.
- [4] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative Filtering for Implicit Feedback Datasets. In *IEEE ICDM*, pages 263–272, 2008.
- [5] Zhu Sun, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, and Cong Geng. Are We Evaluating Rigorously? Benchmarking Recommendation for Reproducible Evaluation and Fair Comparison. In *Proceedings of ACM RecSys*, pages 23–32, 2020. <https://doi.org/10.1145/3383313.3412489>.
- [6] Zhu Sun, Hui Fang, Jie Yang, Xinghua Qu, Hongyang Liu, Di Yu, Yew-Soon Ong, and Jie Zhang. DaisyRec 2.0: Benchmarking Recommendation for Rigorous Evaluation. *IEEE*

- Transactions on Pattern Analysis and Machine Intelligence*, 45(7):8206–8226, 2023. <https://doi.org/10.1109/TPAMI.2022.3231891>.
- [7] Walid Krichene and Steffen Rendle. On Sampled Metrics for Item Recommendation. In *Proceedings of ACM KDD*, 2020.
 - [8] Wayne Xin Zhao et al. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. In *Proceedings of ACM CIKM*, pages 4653–4664, 2021. <https://doi.org/10.1145/3459637.3482016>.
 - [9] Lanling Xu et al. Towards a More User-Friendly and Easy-to-Use Benchmark Library for Recommender Systems. In *Proceedings of ACM SIGIR*, pages 2837–2847, 2023. <https://doi.org/10.1145/3539618.3591889>.
 - [10] RUCAIBox. RecBole 1.2.1 Data Settings Documentation. https://recbole.io/docs/user_guide/config/data_settings.html. Accessed September 2026.
 - [11] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Proceedings of ACM RecSys*, pages 101–109, 2019.
 - [12] Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. A Re-visit of the Popularity Baseline in Recommender Systems. In *Proceedings of ACM SIGIR*, pages 1749–1752, 2020. <https://doi.org/10.1145/3397271.3401233>.
 - [13] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-Based Collaborative Filtering Recommendation Algorithms. In *Proceedings of WWW*, pages 285–295, 2001.
 - [14] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *Proceedings of UAI*, pages 452–461, 2009.
 - [15] Harald Steck. Embarrassingly Shallow Autoencoders for Sparse Data. In *Proceedings of WWW*, 2019.
 - [16] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural Collaborative Filtering. In *Proceedings of WWW*, pages 173–182, 2017.
 - [17] Dawen Liang, Rahul G. Krishnan, Matthew D. Hoffman, and Tony Jebara. Variational Autoencoders for Collaborative Filtering. In *Proceedings of WWW*, pages 689–698, 2018.
 - [18] Balazs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based Recommendations with Recurrent Neural Networks. In *ICLR*, 2016.
 - [19] Wang-Cheng Kang and Julian McAuley. Self-Attentive Sequential Recommendation. In *IEEE ICDM*, pages 197–206, 2018.
 - [20] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of ACM CIKM*, pages 1441–1450, 2019.
 - [21] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of ACM SIGIR*, pages 639–648, 2020.

- [22] Filippo Betello, Federico Siciliano, Pushkar Mishra, and Fabrizio Silvestri. Investigating the Robustness of Sequential Recommender Systems Against Training Data Perturbations. In *Advances in Information Retrieval (ECIR 2024)*, pages 205–220, 2024. https://doi.org/10.1007/978-3-031-56060-6_14.
- [23] Joao Felipe Guedes da Silva, Natanael Nunes de Moura Junior, and Luiz Pereira Caloba. Effects of Data Sparsity on Recommender Systems Based on Collaborative Filtering. In *International Joint Conference on Neural Networks*, 2018. <https://doi.org/10.1109/IJCNN.2018.8489095>.
- [24] Danil Gusak, Anna Volodkevich, Anton Klenitskiy, Alexey Vasilev, and Evgeny Frolov. Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders. In *Proceedings of ACM RecSys*, 2025. <https://doi.org/10.1145/3705328.3748164>.
- [25] Danil Gusak, Nikita Sukhorukov, and Evgeny Frolov. Recommendation Is a Dish Better Served Warm. In *Proceedings of ACM RecSys*, 2025. <https://doi.org/10.1145/3705328.3759331>.